NanoServe - Setup & Usage Guide

NanoServe - Setup & Usage Guide

Complete guide for installing, running, and testing NanoServe in every mode.

Table of contents

Platform status

GPU priority at runtime: **CUDA -> OpenCL -> CPU fallback** (with warnings in API response).

Naming

Prerequisites

Full checklist: REQUIREMENTS.md (non-Docker prior requirements).
Debian/Ubuntu base:
sudo apt-get install -y build-essential cmake curl python3 python3-pip python3-venv git nginx

Native install (no Docker)

git clone https://github.com/<your-org>/NanoServe.git && cd NanoServe
./install.sh # CPU - writes .env.nanoserve
ENABLE_CUDA=1 ./install.sh # + NVIDIA CUDA
ENABLE_OPENCL=1 ./install.sh # + OpenCL
source .venv/bin/activate && source .env.nanoserve
./scripts/run_native.sh # dev / ~150 users - http://localhost:8000
./scripts/run_native_300.sh # production / 300 users (nginx + gunicorn)
See USAGE.md for Web UI, API, SDK, and TUI.
See SCALING.md for env tuning.
Reports: reports/FULL_TEST_REPORT.md.

Python SDK

Install: `pip install -e .`
Quantizer
from nanoserve import Quantizer
import numpy as np
weights = np.random.randn(256, 1024).astype(np.float32)
q, scales = Quantizer.quantize_int8(weights)
Quantizer.write_nanoq("model-int8.nanoq", q, scales, 256, 1024)
CLI:
python -m nanoserve.quantizer --rows 256 --cols 1024 --out model-int8.nanoq
nanoserve-quantizer --out model-int8.nanoq
In-process inference

```
from nanoserve import NanoServe
engine = NanoServe(device="auto")
text = engine.generate("Explain buddy allocators", max_tokens=32)
print(text, engine.last_device, engine.last_warnings)
```

asy

```
import asyncio
async def run():
    engine = NanoServe(device="gpu")
    return await engine.generate_async("Hello", max_tokens=16)
asyncio.run(run())
```

Demo script: `python examples/sdk_demo.py`

HTTP server & Web UI

The Web UI is a single-page app served at `/` by FastAPI. It is **fully working** in native and Docker installs.

```
python server/main.py
```

or: u

- **Web UI:** http://localhost:8000 - textarea prompt, max tokens, **Compute Engine** dropdown (CPU / GPU / Auto)
- **Health:** http://localhost:8000/health - reports `gpu_cuda`, `gpu_opencl`, `gpu_available`
- **Completions:** `POST /v1/completions`

Request body:

```
{
  "prompt": "Hello world",
  "max_tokens": 50,
  "device": "cpu"
}
```

`device` options: `"cpu"` (default), `"gpu"`, `"auto"`.

Terminal UI (TUI)

```
python tui/client.py http://127.0.0.1:8000 --device cpu
```

Interactive commands:

Load test:

```
python tui/load_test.py --users 50 --device auto
```

Device selection

```
flowchart TD
  req[Request device param] --> cpu{device=cpu?}
  cpu --> yes  runCPU[CPU AVX2 pool]
  cpu --> no  gpu{device=gpu?}
  gpu --> yes  tryGPU{CUDA or OpenCL available?}
  tryGPU --> yes  runGPU[GPU pool]
  tryGPU --> no  fallback[CPU + warning]
  gpu --> no auto  autoGPU{GPU available?}
  autoGPU --> yes  runGPU
  autoGPU --> no  runCPU
```

Priority for GPU: **CUDA -> OpenCL -> CPU fallback**.

Docker deployment

```
### CPU (default)
docker compose up --build
### GPU profile
docker compose --profile gpu up --build
```

Runtime images contain only compiled `.so` files and the Python wheel - no CUDA dev toolkit in the final CPU layer.

Building GPU backends

CUDA
Requires NVIDIA driver + CUDA toolkit. Build with:
cd engine/build
cmake .. -DNANOSERVE_ENABLE_CUDA=ON
make -j$(nproc)
Status: Verified working - tiled INT8 GEMV matmul kernel, buddy-allocator host buffers, CPU/CUDA parity on RTX 3060.
CUDA 11.x + GCC 15 requires `engine/nvcc_wrapper.sh` and `g++-9` (configured in CMake when CUDA is enabled).
OpenCL
Requires dev headers at build time:
sudo apt-get install -y ocl-icd-opencl-dev opencl-headers
cd engine/build
cmake .. -DNANOSERVE_ENABLE_OPENCL=ON
make -j$(nproc)
Status: Implementation complete; enable when OpenCL ICD is installed. Falls back to CPU if unavailable.
Verify probes
from nanoserve.engine.worker import EngineWorker
print("CUDA:", EngineWorker.probe_cuda())
print("OpenCL:", EngineWorker.probe_opencl())

C++23 engine

The inference engine is built as **C++23** (`CMAKE_CXX_STANDARD 23` in `engine/CMakeLists.txt`).
Modern features in use:
- `std::span`, `std::string_view`, `std::unique_ptr`, `std::make_unique`
- `enum class EngineBackendKind`
- Designated initializers (`PoolBufferView{}`)
CUDA `.cu` translation units use CUDA C++17 (standard for device code); all host `.cpp` files are C++23.

Testing

1. SDK import test
pip install -e .
python -c "from nanoserve import NanoServe, Quantizer; print('ok')"
2. Quantizer CLI
python -m nanoserve.quantizer --out /tmp/test.nanoq
ls -la /tmp/test.nanoq
3. SDK demo
export LD_LIBRARY_PATH=$(pwd)/allocator/target/release:$LD_LIBRARY_PATH
export NANOSERVE_ENGINE_LIB=$(pwd)/engine/build/libnanoserve_engine.so
python examples/sdk_demo.py
4. API smoke test

star

curl -s http://localhost:8000/health python -m json.tool
curl -s -X POST http://localhost:8000/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"prompt":"hi","max_tokens":8,"device":"cpu"}' python -m json.tool
curl -s -X POST http://localhost:8000/v1/completions \
  -H 'Content-Type: application/json' \
  -d '{"prompt":"hi","max_tokens":8,"device":"gpu"}' python -m json.tool
5. GPU fallback test (no GPU)
Expect `"device": "cpu"` and a warning in `"warnings"`.
6. Load test
python tui/load_test.py --users 20 --device cpu
7. C allocator demo (unchanged)
gcc -std=c17 -O2 -o c_examples/pool_demo c_examples/pool_demo.c \

-Lallocator/target/release -lbuddy_alloc -lpthread -ldl
LD_LIBRARY_PATH=allocator/target/release ./c_examples/pool_demo

Troubleshooting

To generate HTML/PDF locally:
pip install fpdf2
python3 scripts/generate_reports.py

-> d